applied sciences

MDPI

Article

RFP-YOLO26: A Fixed Single-Cycle Feedback Feature Pyramid for Slender Obstacle Detection in Autonomous Street-Sweeping Vehicles

Zhongwen Chen [1], Qingbing Zeng [1,*], Zihua Chen [1], Yixiao Zhang [1], Heng Yang [2] and Qihao Wang [3]

[1] School of Mechanical and Automotive Engineering, Shanghai University of Engineering Science, No. 333, Longteng Road, Shanghai 201620, China; explicit689@163.com (Z.C.); zihuachen@sues.edu.cn (Z.C.); yixiaozhang@sues.edu.cn (Y.Z.)
[2] Hunan Niu En Chi New Energy Vehicle Co., Ltd., Majiahe, Zhuzhou 412007, China; chenhui8044@163.com
[3] School of Mechanical Engineering, Shanghai Industrial Technology School, Xitaizhi Road, Shanghai 200231, China; wjfjia@sina.com
* Correspondence: qingbingzeng@sues.edu.cn

Abstract

Slender obstacles, such as ropes, cables, and rubber hoses, may interfere with the operation of autonomous street-sweeping vehicles because of their narrow shapes and weak visual features. This study presents RFP-YOLO26 as an applied detector-design and systems-integration approach that combines established SPDConv, C3k2_Faster_EMA, and SimAM modules with a fixed single-cycle feedback feature pyramid consisting of an initial top-down pass, one bottom-up feedback pass, and a second top-down refinement pass. Experiments were conducted on the proprietary USLO dataset using a random image-level split. On the current internal test set, RFP-YOLO26 achieved 97.9% mAP@0.5, 65.0% mAP@0.5:0.95, 98.1% precision, and 95.6% recall. Compared with YOLOv26n, these values represent increases of 2.9, 2.2, 1.7, and 4.3 percentage points, respectively. RFP-YOLO26 contains 4.21 M parameters and requires 9.9 GFLOPs, compared with 2.38 M parameters and 5.2 GFLOPs for YOLOv26n. Deployment on the Jetson Orin Nano indicates embedded execution feasibility under the reported configuration. The primary benchmark remains a seed-0 descriptive comparison. The supplementary five-seed analysis and the seed-0 principal-model comparison under route-disjoint Split A retained the same model ordering; the additional route-disjoint runs provided descriptive route-level summaries. However, the available evidence does not establish universal statistical superiority, external generalization, or improved operational safety.

Keywords: autonomous street-sweeping vehicle; slender obstacle detection; YOLO26; fixed single-cycle feedback feature pyramid; SPDConv

Check for updates

Received: 21 June 2026
Revised: 5 August 2026
Accepted: 6 August 2026
Published: 11 August 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

Deep-learning-based object detection has been investigated for wheeled mobile robots and autonomous street-sweeping platforms [1,2]. During operation, slender obstacles such as nylon ropes, electrical cables, and rubber hoses may become entangled in sweeping brushes or obstruct suction inlets, potentially causing equipment damage and operational interruption. As illustrated in Figure 1, these obstacles frequently exhibit narrow widths, elongated shapes, irregular spatial arrangements, and weak contrast with road surfaces. Their timely detection is therefore an important perception task for autonomous street-sweeping vehicles.



Figure 1. Example of a slender obstacle that may interfere with the operation of an autonomous street-sweeping vehicle.

The task considered in this study is single-class axis-aligned bounding-box detection. Slender obstacles can be difficult to detect because their discriminative visual evidence is often concentrated within a small number of pixels across their width. Repeated spatial downsampling may weaken these fine-scale responses, particularly when the apparent object width approaches the sampling stride of the network. Their elongated geometry and variable shape also require the detector to combine fine spatial information with higher-level semantic features across multiple scales. In addition, cracks, pavement joints, shadows, lane markings, and illumination variations may produce visually similar linear patterns and increase the difficulty of distinguishing obstacles from the surrounding background. Related studies have addressed weak small-object features in lightweight detectors and topology-aware representation of tubular structures [3,4]. Dynamic Snake Convolution is formulated as a segmentation method rather than an axis-aligned object detector and is therefore considered related structural-modeling work rather than a direct detection baseline.

Multi-scale feature fusion has been widely used to address scale variation in object detection. Feature Pyramid Networks construct a top-down pathway with lateral connections to combine semantic and spatial information [5]. PANet introduces bottom-up path augmentation to further propagate lower-level localization features [6]. EfficientDet proposes BiFPN, which uses repeated bidirectional fusion and learnable feature-fusion weights [7]. DetectoRS introduces a Recursive Feature Pyramid that feeds feature-pyramid outputs back into the backbone for another backbone–pyramid processing stage [8]. These architectures provide important foundations for multi-scale detection, but their information-flow topologies differ from the fixed feature-feedback structure evaluated in the present study.

Several established modules have also been proposed to improve downsampling, feature extraction, and attention. SPDConv applies a space-to-depth rearrangement before non-strided convolution to retain local spatial samples explicitly during downsampling [9]. FasterNet introduces partial convolution, which performs spatial convolution on only a

subset of the input channels [10]. Efficient Multi-Scale Attention combines grouped feature processing, directional pooling, and cross-spatial interaction [11], while SimAM generates three-dimensional attention weights without additional learnable parameters [12]. These modules are established operators. The contribution of this study does not lie in proposing them individually, but in their implementation within the proposed detector and in the fixed single-cycle feature-feedback topology used to connect the multi-scale features.

This study proposes RFP-YOLO26, a slender-obstacle detector developed from YOLOv26n [13]. The proposed neck is a fixed single-cycle feedback feature pyramid consisting of an initial top-down fusion pass, one bottom-up feedback pass, and a second top-down refinement pass. Relative to the PAN-like baseline neck, which terminates after bottom-up aggregation, the proposed topology uses the feedback-enhanced low-resolution features to perform an additional top-down refinement. The modules in the different stages are independently instantiated, and no weights are shared between the two top-down passes. Unlike DetectoRS-style RFP, the proposed structure does not feed pyramid features back through the backbone and does not perform an unspecified number of iterative recursive passes.

SPDConv is used in selected downsampling stages to rearrange local spatial samples before learned channel projection. C3k2_Faster_EMA modules are employed for feature extraction and fusion, and parameter-free SimAM modules are applied to the three output feature scales before the detection head. These components are evaluated as part of the complete detector using bounding-box metrics. The study does not use segmentation masks or centerline annotations and therefore does not claim direct measurement of object continuity or pixel-level structural completeness.

Accordingly, the present work is positioned as an applied detector-design and systems-integration study rather than as a fundamental new detection theory. Its contribution lies in the implementation and evaluation of a fixed single-cycle feature-feedback topology, the task-oriented integration of established operators, and the platform-specific evaluation of the resulting detector on an autonomous street-sweeping vehicle.

The main contributions of this study are summarized as follows:

1. A fixed single-cycle feature-feedback topology is implemented and evaluated for multi-scale slender-obstacle detection. It extends the PAN-like bidirectional baseline with one additional top-down refinement pass after the bottom-up feedback stage. The modules in the different stages are independently instantiated without weight sharing.
2. Established SPDConv, C3k2_Faster_EMA, and SimAM operators are integrated into the detector to modify downsampling, feature extraction, feature fusion, and output-scale weighting. The study does not claim the individual invention of these operators; instead, their tensor mappings, execution sequence, and complete-model complexity are documented according to the implemented architecture. The experimental results are interpreted at the level of the integrated detector and are not used to attribute model-level performance improvements to any individual operator.
3. The resulting detector is evaluated on the proprietary 2000-image USLO dataset and deployed on the Jetson Orin Nano. The results are interpreted as descriptive internal evidence of an accuracy–complexity trade-off and platform-specific embedded execution feasibility, rather than as proof of statistical superiority, general lightweight performance, or improved vehicle safety.

2. Materials and Methods

2.1. Related Object-Detection Methods

Object-detection methods are commonly divided into two-stage and single-stage frameworks. Two-stage detectors, such as Faster R-CNN, first generate candidate regions

and subsequently perform classification and bounding-box regression [14]. These methods can provide accurate localization, but their multi-stage processing pipeline may introduce additional computational and memory overhead. Single-stage detectors, including the YOLO family, perform object classification and bounding-box regression within a unified prediction framework [15,16]. Their direct prediction structure has contributed to their widespread use in mobile and embedded visual-perception applications. Recent real-time object detectors have further explored improvements in feature aggregation, information preservation, and end-to-end prediction. Gold-YOLO introduces a gather-and-distribute mechanism for multi-scale feature interaction [17]. YOLOv9 combines programmable gradient information with a generalized efficient layer-aggregation network [18], whereas YOLOv10 adopts consistent dual assignments and an efficiency–accuracy oriented design for end-to-end detection [19]. Transformer-based real-time detectors have also been developed. RT-DETR uses an efficient hybrid encoder for multi-scale processing [20], and D-FINE reformulates bounding-box regression as fine-grained distribution refinement [21]. These methods represent recent developments in real-time object detection but are discussed as related work rather than as directly evaluated comparators in the present study.

Edge deployment requires consideration of parameter count, computational cost, memory access, operator support, numerical precision, and end-to-end latency. Existing approaches reduce model cost through hardware-aware compact backbones, inexpensive feature-generation operations, partial convolution, pruning, and efficient feature-fusion structures [22–24]. More recent mobile-oriented architectures continue to emphasize that theoretical computation and measured hardware latency should be considered jointly. RepViT incorporates efficient architectural elements associated with lightweight vision transformers into a convolutional network intended for resource-constrained devices [25]. MobileNetV4 introduces hardware-oriented building blocks and architecture-search strategies for mobile CPUs, GPUs, DSPs, and dedicated accelerators [26]. Recent lightweight detector research, such as YOLO-ULM, has also investigated efficient feature aggregation and downsampling for real-time object detection [27]. These studies motivate the separate reporting of parameter count, GFLOPs, and measured embedded throughput in the present work. However, a low parameter count alone does not guarantee high embedded throughput, because runtime performance also depends on hardware utilization, memory movement, software frameworks, and operator implementation.

The present study adopts YOLOv26n as the baseline detector [13] and modifies its backbone and multi-scale feature-aggregation neck for single-class axis-aligned bounding-box detection of slender obstacles. The proposed RFP-YOLO26 model is evaluated as an accuracy–complexity trade-off rather than as a lower-compute replacement for YOLOv26n. Its accuracy and computational cost are evaluated jointly in the experimental section.

Slender obstacles, including ropes, cables, and rubber hoses, often occupy only a narrow transverse region in an image. Their visible appearance may also be affected by curvature, occlusion, illumination, distance, and similarity to pavement cracks, joints, shadows, or lane markings. Repeated spatial downsampling may weaken their fine-scale feature responses. The methodological objective of this study is therefore to evaluate the bounding-box detection performance of an integrated detector combining modified downsampling, feature extraction, multi-scale feedback, and attention under the current experimental protocol. The study does not seek to estimate the independent effect of each incorporated component.

2.2. Multi-Scale Feature Fusion and Fixed Feedback Topologies

Multi-scale feature fusion is widely used to combine high-level semantic information with higher-resolution spatial information. Feature Pyramid Networks construct a top-

down pathway with lateral connections between backbone features [5]. PANet extends this design by adding a bottom-up path-augmentation stage, thereby shortening the information path between lower-level localization features and higher-level representations [6]. EfficientDet introduces a weighted bidirectional feature pyramid network, referred to as BiFPN, which repeatedly performs top-down and bottom-up fusion using learnable feature-fusion weights [7].

DetectoRS proposes a Recursive Feature Pyramid in which feature-pyramid outputs are fed back into the bottom-up backbone for another backbone–pyramid processing stage [8]. This mechanism represents a genuine backbone-level recursive design.

The YOLOv26n baseline used in the present study already employs a PAN-like bidirectional neck. It first performs top-down feature fusion and then constructs a bottom-up aggregation pathway. Therefore, the baseline is not treated as a purely top-down FPN.

The proposed feature-fusion topology differs from both the baseline neck and DetectoRS-style RFP. It consists of three fixed stages: (1) an initial top-down feature-fusion pass; (2) one bottom-up feedback pass; and (3) a second top-down refinement pass.

The modules used in these stages are independently instantiated, and weights are not shared between the two top-down passes. The structure is executed once during each forward pass. It does not perform an unspecified number of recurrent iterations, does not use a convergence criterion, and does not feed the pyramid outputs through the backbone again. It is therefore referred to throughout this manuscript as a fixed single-cycle feedback feature pyramid.

The purpose of the additional top-down refinement is to reuse the features obtained after the bottom-up feedback stage when constructing the final high-resolution detection features. The performance of the cumulative configuration containing this additional refinement is reported in the progressive component-integration study. That study describes the model-development trajectory and is not used to estimate an independent effect of the refinement stage.

In addition, the effect of feedback depth is evaluated using configurations with zero, one, and two feedback cycles. Feedback-0 contains only the initial top-down fusion stage. Feedback-1 corresponds to the proposed fixed single-cycle topology, consisting of an initial top-down pass, one bottom-up feedback pass, and a second top-down refinement pass. Feedback-2 adds a second independently instantiated bottom-up feedback and top-down refinement cycle.

The modules in the different stages and cycles are independently instantiated, and weights are not shared. Because each additional feedback cycle introduces extra downsampling and feature-fusion modules, the zero-, one-, and two-cycle configurations are not matched in parameter count or computational cost.

The descriptions above clarify the information-flow differences among the related feature-pyramid architectures. To provide a controlled comparison with an alternative feature-pyramid design, a BiFPN-style control was additionally implemented. The control retained the same backbone, SPDConv layers, C3k2_Faster_EMA modules, SimAM modules, detection head, dataset partition, training settings, checkpoint-selection procedure, and evaluation protocol as RFP-YOLO26, while replacing the fixed single-cycle feedback neck with a single weighted bidirectional fusion neck. The complete seed-0 comparison is reported in Supplementary Table S4. The experiment is interpreted as a descriptive architecture-level comparison under a common detector framework and a closely aligned computational budget.

2.3. Efficient Feature Extraction and Attention Mechanisms

Embedded inference performance depends not only on theoretical FLOPs but also on memory access and the efficiency of individual operators. FasterNet introduces partial convolution, or PConv, which applies a spatial convolution to only a subset of the feature channels while the remaining channels follow an identity path [10]. In the implementation used in this study, one quarter of the channels are processed by a 3×3 partial convolution. The resulting feature is subsequently processed by two pointwise convolutions with the channel mapping $D \rightarrow 2D \rightarrow D$.

The one-sixteenth complexity ratio associated with PConv applies only to the isolated 3×3 partial-convolution operator when one quarter of the channels are processed. It does not represent the cost of the complete feature-extraction block, which additionally includes pointwise convolutions, normalization, activation, attention, residual aggregation, and the input and output projections inherited from the surrounding C3k2 structure.

Lightweight attention mechanisms seek to improve feature weighting without introducing large attention subnetworks. For example, ECA-Net models local cross-channel interaction using a lightweight one-dimensional convolution [28]. Efficient Multi-Scale Attention, abbreviated as EMA in this manuscript, divides the input channels into groups and combines directional pooling, local convolution, and cross-spatial interaction to generate spatial feature weights [11]. EMA refers exclusively to Efficient Multi-Scale Attention; it does not refer to an exponential moving average of model parameters.

SimAM is a parameter-free attention mechanism that estimates three-dimensional neuron importance using an energy-function formulation [12]. In RFP-YOLO26, SimAM is applied to the final P_3, P_4, and P_5 feature maps before they are passed to the detection head.

The C3k2_Faster_EMA module used in this study integrates the partial-convolution-based Faster block with Efficient Multi-Scale Attention inside the outer C3k2 feature-extraction structure. PConv, EMA, and SimAM are established operators; the present contribution does not claim their individual invention. Their implementation, execution order, tensor dimensions, and contribution within the complete detector are described in the following model-architecture section.

2.4. Slender-Obstacle Detection and Spatial Downsampling

Several operators have been investigated for narrow, small, or deformable visual structures. Lightweight small-object detectors seek to retain weak fine-scale responses under constrained computation [3]. Dynamic Snake Convolution uses dynamically adjusted sampling locations to model curved tubular structures [4]. However, Dynamic Snake Convolution is designed for segmentation and is not used as a direct object-detection comparator in this study.

SPD-Conv was introduced as a downsampling alternative for low-resolution images and small objects [9]. Its space-to-depth stage rearranges the samples from each local spatial neighborhood along the channel dimension before a non-strided convolution is applied. For an input tensor

$$X \in \mathbb{R}^{C_{\text{in}} \times H \times W}, \tag{1}$$

a scale-2 space-to-depth rearrangement produces

$$X_{\text{s2d}} \in \mathbb{R}^{4C_{\text{in}} \times \frac{H}{2} \times \frac{W}{2}}. \tag{2}$$

A subsequent 3×3 convolution with stride 1 and padding 1 maps the rearranged feature to

$$Y \in \mathbb{R}^{C_{\text{out}} \times \frac{H}{2} \times \frac{W}{2}}. \tag{3}$$

In the downsampling stages used in RFP-YOLO26,

$$C_{\text{out}} = 2C_{\text{in}}. \tag{4}$$

The space-to-depth operation itself is a deterministic tensor rearrangement. However, the subsequent convolution performs learned channel mixing and projection. Consequently, the complete SPDConv module is not described as strictly information-lossless or pixel-lossless.

In RFP-YOLO26, selected stride-2 convolutional downsampling layers after the initial stem are replaced by SPDConv. The purpose is to present the subsequent convolution with an explicit channel-wise arrangement of the samples from each 2×2 neighborhood before spatial resolution is reduced. The complete detector incorporating this design is evaluated using axis-aligned bounding-box metrics on the USLO dataset. The reported cumulative experiment does not isolate the independent effect of SPDConv.

The present study does not use segmentation masks, centerline annotations, or continuity-sensitive metrics. Therefore, it does not claim that SPDConv or the complete model directly preserves pixel-level object continuity or structural completeness. The reported outcomes are limited to bounding-box detection performance under the original random-split protocol and the supplementary route-disjoint protocol.

Based on the above considerations, RFP-YOLO26 combines four implementation components: (1) SPDConv for selected spatial-downsampling stages; (2) C3k2_Faster_EMA for feature extraction and feature fusion; (3) a fixed single-cycle feedback feature pyramid for multi-scale aggregation; and (4) SimAM on the final three detection scales.

The complete architecture is evaluated as an accuracy–complexity trade-off. The use of these components does not imply that the resulting model has lower computational cost than the YOLOv26n baseline, and embedded execution is assessed separately under the reported Jetson Orin Nano configuration.

3. RFP-YOLO26 Detection Framework

3.1. Overall Architecture

RFP-YOLO26 is developed from YOLOv26n for the single-class axis-aligned bounding-box detection of slender obstacles in autonomous street-sweeping environments. As illustrated in Figure 2, the network consists of a backbone, a fixed single-cycle feedback feature pyramid, three SimAM modules, and a multi-scale detection head.

The initial stem remains a conventional 3×3 convolution with stride 2. Selected subsequent stride-2 downsampling layers are replaced by SPDConv modules. C3k2_Faster_EMA modules are used in the backbone and feature-fusion neck, while SimAM is applied to the final three feature scales before detection.

The YOLOv26n baseline used in this study contains a PAN-like bidirectional neck with an initial top-down fusion pathway followed by a bottom-up aggregation pathway. RFP-YOLO26 extends this structure with one additional top-down refinement pass. The resulting topology therefore consists of: (1) an initial top-down fusion pass; (2) one bottom-up feedback pass; and (3) a second top-down refinement pass.

The modules at different stages are independently instantiated, and weights are not shared between the two top-down passes. The complete topology is executed once during each forward pass. It does not perform recurrent refinement until convergence and does not feed feature-pyramid outputs back through the backbone, as in the DetectoRS Recursive Feature Pyramid [8].

The final feature maps are supplied to a detection head that performs classification and axis-aligned bounding-box regression. The detector does not generate segmentation masks or centerlines. Because the additional feedback and refinement paths increase com-

putational cost, RFP-YOLO26 is evaluated as an accuracy–complexity trade-off rather than as a lower-compute replacement for YOLOv26n.

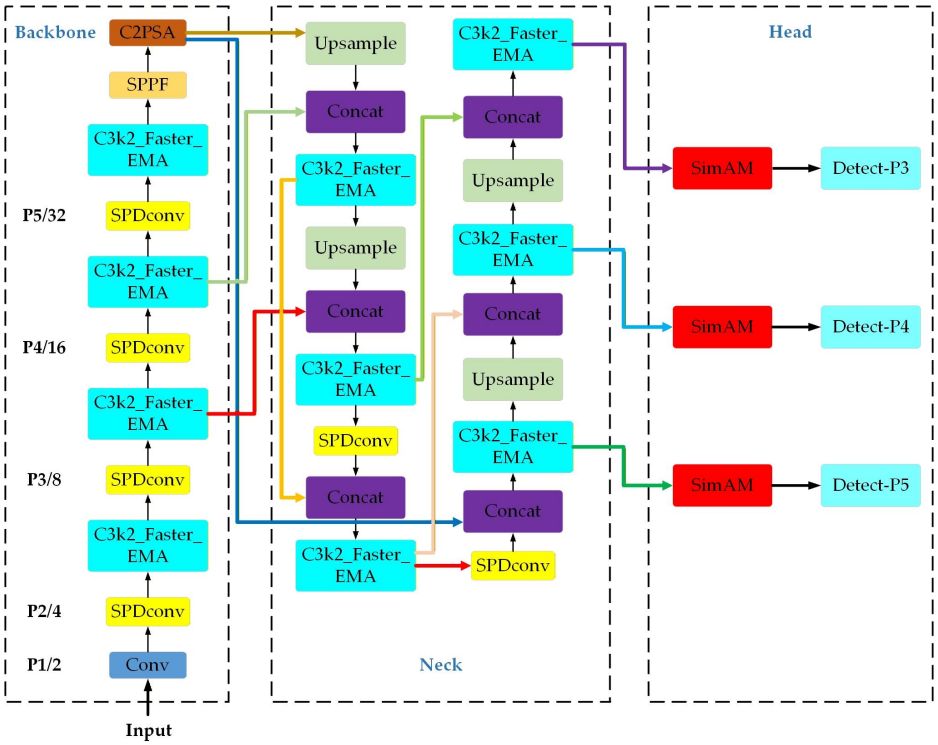


Figure 2. Overall architecture of RFP-YOLO26. The backbone employs SPDConv and C3k2_Faster_EMA modules. The neck consists of an initial top-down fusion pass, one bottom-up feedback pass, and a second top-down refinement pass. SimAM is applied to the final three feature scales before axis-aligned bounding-box detection.

3.2. SPDConv Downsampling

A stride-2 convolution reduces the height and width of a feature map, but it should not be described as directly discarding 75% of the input pixels because each output activation is produced through learned convolutional aggregation over a local receptive field.

For slender obstacles occupying only a small number of pixels across their width, repeated spatial reduction may weaken fine-scale feature responses. RFP-YOLO26 therefore replaces selected stride-2 downsampling layers after the initial stem with SPDConv [9]. The structure of the implemented SPDConv module is shown in Figure 3.

For an input tensor

$$X \in \mathbb{R}^{C_{\text{in}} \times H \times W}, \tag{5}$$

a scale-2 space-to-depth operation rearranges the four spatial positions in each 2×2 neighborhood along the channel dimension:

$$X_{\text{s2d}} = \text{S2D}_2(X) \in \mathbb{R}^{4C_{\text{in}} \times \frac{H}{2} \times \frac{W}{2}}. \tag{6}$$

A 3×3 convolution with stride 1 and padding 1 then performs learned channel mixing and projection, followed by batch normalization and SiLU activation:

$$Y = \text{SiLU}\left[\text{BN}\left(\text{Conv}_{3\times3,s=1,p=1}^{4C_{\text{in}} \to C_{\text{out}}}(X_{\text{s2d}})\right)\right]. \tag{7}$$

In the implemented downsampling stages, $C_{out} = 2C_{in}$. The complete tensor mapping is

$$C_{in} \times H \times W \rightarrow 4C_{in} \times \frac{H}{2} \times \frac{W}{2} \rightarrow 2C_{in} \times \frac{H}{2} \times \frac{W}{2}. \tag{8}$$

The space-to-depth operation is a deterministic tensor rearrangement, whereas the subsequent convolution performs learned projection from $4C_{in}$ to C_{out}. The complete SPDConv module is therefore not described as strictly information-lossless or pixel-lossless.

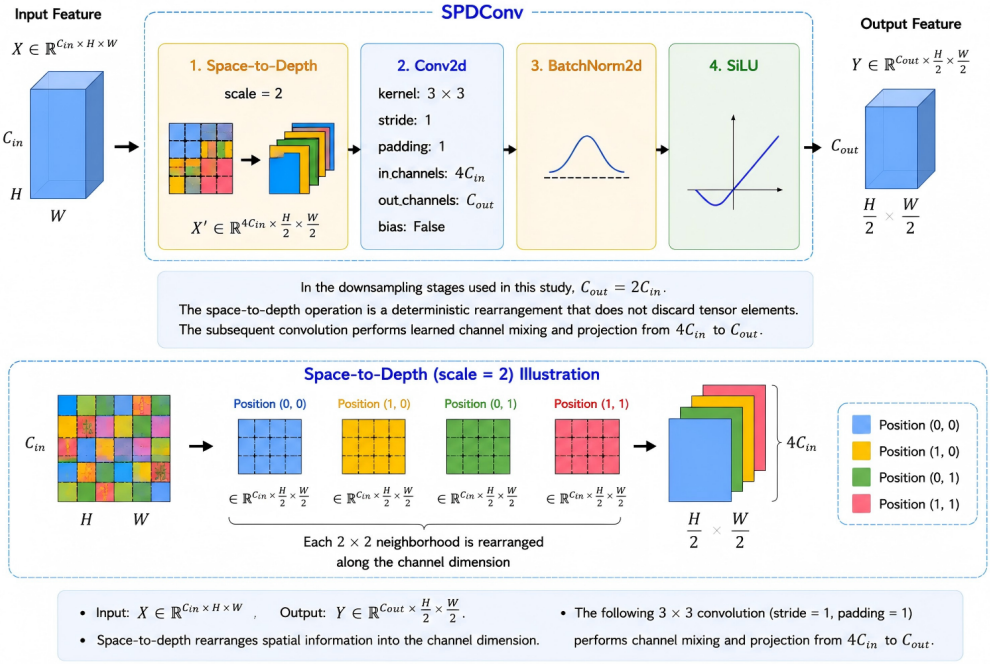


Figure 3. Structure of the SPDConv downsampling module. A scale-2 space-to-depth operation rearranges the input from $C_{in} \times H \times W$ to $4C_{in} \times H/2 \times W/2$. A 3×3 convolution with stride 1 and padding 1 then projects the feature to $C_{out} \times H/2 \times W/2$, followed by batch normalization and SiLU activation. In the implemented downsampling stages, $C_{out} = 2C_{in}$.

3.3. C3k2_Faster_EMA Feature-Extraction Module

The C3k2_Faster_EMA module retains the feature projection, branch-splitting, branch-processing, concatenation, and output projection structure of C3k2. Its internal units combine partial convolution, a two-layer pointwise convolutional MLP, DropPath, Efficient Multi-Scale Attention, and residual addition. The structure of the C3k2_Faster_EMA module and its internal Faster_Block_EMA is shown in Figure 4.

For an input feature

$$X \in \mathbb{R}^{D \times H \times W}, \tag{9}$$

the implementation uses `n_div` $= 4$. One quarter of the channels are processed by a 3×3 partial convolution, while the remaining channels follow an identity path:

$$Z = \mathrm{Concat}\left[\mathrm{Conv}_{3\times3}(X_p), X_i\right], \quad X_p \in \mathbb{R}^{\frac{D}{4} \times H \times W}, \quad X_i \in \mathbb{R}^{\frac{3D}{4} \times H \times W}. \tag{10}$$

The concatenated feature is processed by a pointwise convolutional MLP with `mlp_ratio` $= 2$, corresponding to

$$D \rightarrow 2D \rightarrow D. \tag{11}$$

The first pointwise convolution is followed by batch normalization and SiLU activation, while the second pointwise convolution restores the original channel dimension.

3.3.1. Efficient Multi-Scale Attention and Residual Path

In this manuscript, EMA denotes only Efficient Multi-Scale Attention [11]; it does not refer to an exponential moving average of model parameters. For an input tensor

$$V \in \mathbb{R}^{B\times D\times H\times W}, \tag{12}$$

the implemented EMA module divides the channels into $G = 8$ groups:

$$V_g \in \mathbb{R}^{BG\times \frac{D}{G}\times H\times W}. \tag{13}$$

Directional descriptors are obtained by pooling along the height and width dimensions. A shared 1×1 convolution produces directional weights, while a parallel 3×3 convolution extracts local spatial features. The two branches are combined to generate cross-spatial weights and produce an output with the same dimensions as the input:

$$\mathrm{EMA}(V) \in \mathbb{R}^{B\times D\times H\times W}. \tag{14}$$

The execution sequence of the internal Faster_Block_EMA can be summarized as

$$Y = X + \mathrm{EMA}[\mathrm{DropPath}(\mathcal{M}(\mathrm{PConv}(X)); p = 0.1)], \tag{15}$$

where $\mathcal{M}(\cdot)$ denotes the two-layer pointwise MLP. DropPath is active during training and becomes an identity mapping during evaluation. The LayerScale branch is not activated in the reported configuration.

PConv, FasterNet, and EMA are established operators, and this study does not claim their individual invention.

3.3.2. Operator-Level Complexity

For the isolated 3×3 partial convolution, processing one quarter of the channels produces the ratio

$$\frac{\mathrm{MACs}_{\mathrm{PConv}}}{\mathrm{MACs}_{\text{full-}3\times3}} = \frac{1}{16}. \tag{16}$$

After including the two pointwise convolutions in the $D \to 2D \to D$ MLP, the corresponding ratio relative to one full D-to-D 3×3 convolution is

$$\frac{\mathrm{MACs}_{\mathrm{PConv+MLP}}}{9D^2HW} = \frac{73}{144} \approx 0.507. \tag{17}$$

These ratios apply only to the stated operators. They exclude EMA, normalization, activation, DropPath, residual aggregation, repeated internal blocks, and the outer C3k2 projections, and therefore do not represent the cost of the complete C3k2_Faster_EMA module.

3.4. Fixed Single-Cycle Feedback Feature Pyramid

Let B_3, B_4, and B_5 denote the backbone features at spatial resolutions of 1/8, 1/16, and 1/32 of the input resolution, respectively. Let $\mathcal{U}(\cdot)$ denote nearest-neighbor upsampling by a factor of two, $\mathcal{D}(\cdot)$ denote SPDConv downsampling, and $\mathcal{F}_l^{(k)}(\cdot)$ denote an independently instantiated C3k2_Faster_EMA fusion module at scale l and stage k.

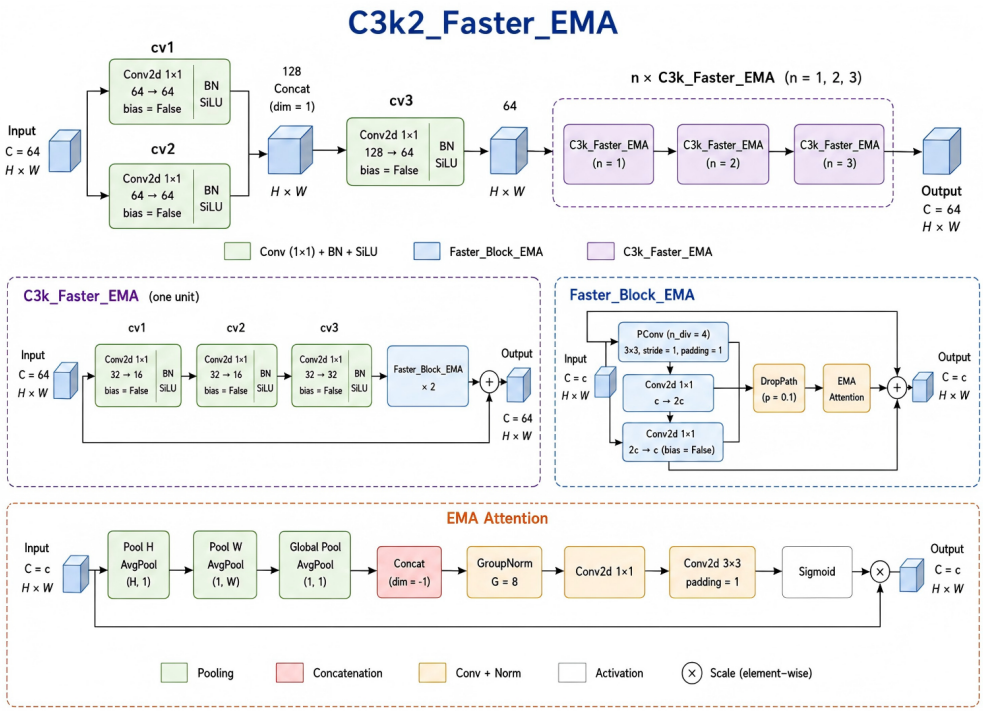


Figure 4. Structure of C3k2_Faster_EMA and its internal Faster_Block_EMA. Within Faster_Block_EMA, one quarter of the channels are processed by a 3×3 partial convolution. The resulting feature is processed by a $D \to 2D \to D$ pointwise MLP, DropPath, Efficient Multi-Scale Attention, and residual addition. The implemented EMA group number is $G = 8$.

3.4.1. Initial Top-Down Fusion

The first top-down pass produces

$$\begin{aligned} T_4 &= \mathcal{F}_4^{(1)}[\mathrm{Concat}(\mathcal{U}(B_5), B_4)], \\ T_3 &= \mathcal{F}_3^{(1)}[\mathrm{Concat}(\mathcal{U}(T_4), B_3)]. \end{aligned} \tag{18}$$

3.4.2. Bottom-Up Feedback

The high-resolution feature T_3 is propagated toward the lower-resolution levels:

$$\begin{aligned} F_4 &= \mathcal{F}_4^{(\mathrm{fb})}[\mathrm{Concat}(\mathcal{D}(T_3), T_4)], \\ F_5 &= \mathcal{F}_5^{(\mathrm{fb})}[\mathrm{Concat}(\mathcal{D}(F_4), B_5)]. \end{aligned} \tag{19}$$

Here, T_4 denotes the feature generated by the initial top-down pass rather than the original backbone feature B_4.

3.4.3. Second Top-Down Refinement

The feedback-enhanced features are used in one additional top-down pass:

$$\begin{aligned} R_4 &= \mathcal{F}_4^{(2)}[\mathrm{Concat}(\mathcal{U}(F_5), F_4)], \\ R_3 &= \mathcal{F}_3^{(2)}[\mathrm{Concat}(\mathcal{U}(R_4), T_3)]. \end{aligned} \tag{20}$$

The final features supplied to the attention modules and detection head are

$$O_3 = \mathrm{SimAM}(R_3), \qquad O_4 = \mathrm{SimAM}(R_4), \qquad O_5 = \mathrm{SimAM}(F_5). \tag{21}$$

The term "feedback" refers to the explicitly executed bottom-up feature propagation followed by one additional top-down refinement pass. The structure is fixed and is executed once per forward pass. Modules in the two top-down stages are independently instantiated and do not share weights. The complete fixed single-cycle feedback feature pyramid is illustrated in Figure 5.

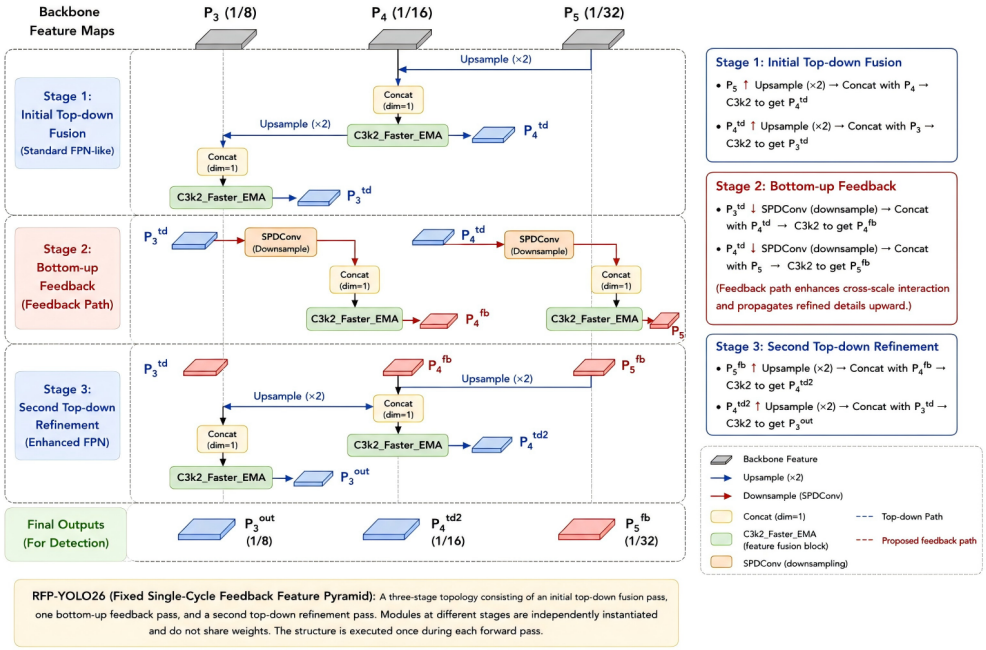


Figure 5. Fixed single-cycle feedback feature pyramid used in RFP-YOLO26. The topology consists of an initial top-down pass ($B_5 \rightarrow T_4 \rightarrow T_3$), one bottom-up feedback pass ($T_3 \rightarrow F_4 \rightarrow F_5$), and a second top-down refinement pass ($F_5 \rightarrow R_4 \rightarrow R_3$). Modules at different stages are independently instantiated and do not share weights.

3.4.4. Distinction from the Baseline and Related Pyramids

The YOLOv26n baseline already contains a PAN-like top-down and bottom-up feature-aggregation neck. The proposed topology differs from the baseline by using the bottom-up features F_4 and F_5 in a second top-down refinement pass before detection.

The proposed structure also differs from BiFPN because it does not use repeated bidirectional cells or learnable feature-fusion weights. Unlike the DetectoRS Recursive Feature Pyramid, it does not feed pyramid outputs back through the backbone for another backbone–pyramid processing stage.

To provide a controlled alternative-neck comparison, a BiFPN-style control was constructed using the same backbone, task-oriented modules, detection head, dataset partition, training settings, checkpoint-selection procedure, and evaluation protocol as RFP-YOLO26. The control replaced the fixed single-cycle feedback neck with a single weighted bidirectional fusion neck and did not contain the proposed second top-down feedback-refinement path.

The BiFPN-control contained 4.13 M parameters and required 9.27 GFLOPs, compared with 4.21 M parameters and 9.90 GFLOPs for RFP-YOLO26. The two configurations were therefore treated as approximately, rather than strictly, compute-matched. The complete seed-0 comparison is reported in Supplementary Table S4.

3.5. SimAM Parameter-Free Attention

SimAM estimates neuron importance using an energy-function formulation without introducing trainable parameters [12]. The structure of the parameter-free SimAM attention module is shown in Figure 6. For an input feature map

$$X \in \mathbb{R}^{B \times C \times H \times W}, \tag{22}$$

the channel-wise spatial mean and variance are

$$\mu_c = \frac{1}{HW} \sum_{i=1}^{H} \sum_{j=1}^{W} X_{c,i,j}, \qquad \sigma_c^2 = \frac{1}{HW-1} \sum_{i=1}^{H} \sum_{j=1}^{W} (X_{c,i,j} - \mu_c)^2. \tag{23}$$

Using $\lambda = 10^{-4}$, the attention response is

$$A_{c,i,j} = \text{Sigmoid}\left[\frac{(X_{c,i,j} - \mu_c)^2}{4(\sigma_c^2 + \lambda)} + \frac{1}{2} \right], \qquad Y = X \odot A. \tag{24}$$

SimAM is applied independently to the final P_3, P_4, and P_5 feature maps before the detection head. It adds no trainable parameters but still requires element-wise arithmetic and sigmoid operations. Its role is evaluated within the cumulative component-integration experiment; no independent claim of causal improvement or module synergy is made.

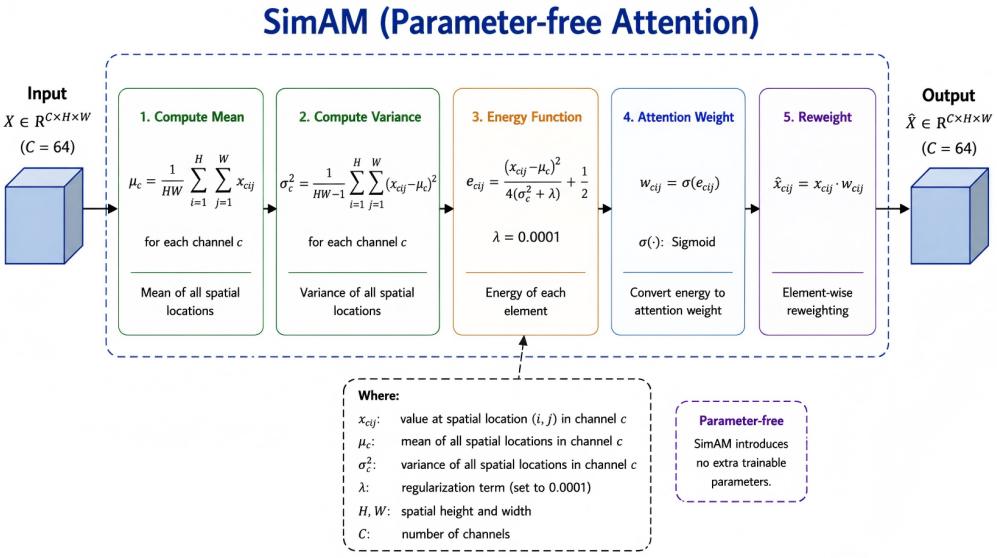


Figure 6. Structure of the parameter-free SimAM attention module. SimAM computes neuron-wise attention responses from the deviation of each activation from its channel mean and applies the resulting weights to the input feature map.

3.6. Complete-Model Complexity

The parameter count and GFLOPs of the complete models were measured using the same profiling procedure with an input resolution of 640 × 640 and a batch size of 1. YOLOv26n contains 2.38 M parameters and requires 5.2 GFLOPs, whereas RFP-YOLO26 contains 4.21 M parameters and requires 9.9 GFLOPs.

RFP-YOLO26 therefore adds 1.83 M parameters and 4.7 GFLOPs. The additional cost mainly arises from the feedback downsampling modules, the independently instantiated feature-fusion modules, and the second top-down refinement pass. The increase is not described as slight, minimal, or negligible. Embedded throughput is reported separately under the Jetson Orin Nano experimental configuration.

4. Experiments

This section describes the USLO dataset, annotation protocol, dataset partition, training and evaluation settings, complexity profiling, on-vehicle runtime configuration, assessment criteria, and reproducibility materials. The primary benchmark, progressive component-integration experiments, and feedback-cycle experiments were conducted using the original random image-level partition of the USLO dataset. A supplementary five-seed analysis was conducted for YOLOv26n and RFP-YOLO26 under the same random-split protocol. An extended route-disjoint evaluation used R01 and R02 for training and two complementary R03–R04 validation–test assignments, with two independently trained random-seed runs evaluated for each held-out test route.

4.1. USLO Dataset

The USLO dataset was constructed for the single-class axis-aligned bounding-box detection of slender obstacles in autonomous street-sweeping environments. It contains 2000 road-scene images with an original resolution of 1920 × 1080 pixels. Data acquisition was conducted from 1 May to 20 May 2026 along four routes anonymized as R01–R04.

The autonomous sweeping vehicle, camera equipment, testing facilities, road-scene data, and field engineering assistance were provided as non-financial, in-kind resources by Hunan Niu En Chi New Energy Vehicle. The company provided no direct financial funding for this study. The study design, annotation, model development, training, evaluation, result interpretation, and manuscript preparation were conducted by the author team.

The road-scene images and experimental-setup photographs included in this manuscript and the Supplementary Materials were captured by the author team and are used with the team's authorization. Road-scene recording was permitted at all acquisition locations and did not require special-access authorization. No identifiable personal information, such as clearly recognizable faces or legible vehicle license plates, is present in the images included in the manuscript or Supplementary Materials.

Images were captured using a UC70 color camera at 1920 × 1080 pixels. The dataset contains independently captured still images and frames extracted from recorded videos. For the video recordings, one frame was sampled every 3 s to reduce redundancy between immediately adjacent frames. Detailed optical calibration parameters were not used in the reported detection experiments.

The four acquisition routes contain 1026, 368, 324, and 282 images, respectively. The distribution of images by acquisition route, weather condition, and illumination condition is summarized in Table 1. The route identifiers indicate the acquisition sources. They were not used to define the original random image-level partition but were used to construct the additional route-disjoint evaluation described in Section 4.3.

Table 1. Distribution of USLO images by acquisition route, weather condition, and illumination condition.

Route	Sunny	Cloudy	Light Rain	Daytime	Twilight	Nighttime	Total
R01	562	346	118	590	256	180	1026
R02	202	123	43	212	92	64	368
R03	178	108	38	186	81	57	324
R04	158	93	31	162	71	49	282
Total	1100	670	230	1150	500	350	2000

Weather and illumination are separate descriptive dimensions. Their counts therefore each independently sum to the corresponding route total.

The scenes contain cord-like objects, including ropes, electrical wires, and network cables; tubular objects, including rubber hoses and plastic tubes; and strip-like elongated

materials. These descriptions characterize physical appearance and do not represent separate detection classes. All annotated obstacles were assigned to one class, denoted as *line*, because the task is to detect the presence and location of potentially obstructive slender objects rather than to classify their materials or functions.

4.2. Annotation Protocol and Label-File Audit

Three annotators participated in the annotation process using LabelMe [29]. All objects were represented by axis-aligned rectangular bounding boxes. The annotations were converted to the standard normalized YOLO format containing the class index, bounding-box center coordinates, width, and height.

For a curved obstacle, one axis-aligned rectangle was used to enclose its complete visible extent. Consequently, the box of a strongly curved object may include surrounding road background. The annotations should therefore be interpreted as bounding-box labels rather than segmentation masks, oriented boxes, or centerline annotations.

The final annotations were manually reviewed before conversion and model training. However, the original annotation workflow did not preserve independent duplicate annotations or a formal adjudication log. Formal inter-annotator agreement cannot therefore be reconstructed retrospectively. Possible ambiguity in curved, partially occluded, overlapping, truncated, or weakly visible objects is acknowledged as a dataset limitation.

An automated audit was performed on the converted YOLO-format label files. The audit examined the number of label files, total bounding-box count, per-image instance count, class indices, empty label files, and malformed annotation rows. The results are summarized in Table 2.

Table 2. Summary of the USLO annotation-file audit.

Audit Item	Value
Number of images	2000
Number of label files	2000
Total bounding-box instances	8278
Average instances per image	4.14
Minimum instances per image	1
Maximum instances per image	11
Empty label files	0
Instances assigned to class 0	8278
Malformed annotation rows	0

4.3. Dataset Partitions and Cross-Subset Audit

The primary experiments used the original random image-level partition of the 2000-image USLO dataset. This partition contained 1400 training images, 400 validation images, and 200 test images, as shown in Table 3. The training subset was used for parameter optimization, the validation subset was used for checkpoint selection, and the test subset was used only for final metric calculation. The multiple-seed variability analysis of YOLOv26n and RFP-YOLO26 was also conducted using this original partition.

The original partition was not grouped by acquisition route, recording date, physical scene, source video, or recording session. Because the dataset contains both independently captured still images and frames extracted from videos, temporally or visually related observations could potentially have been assigned to different subsets. Results obtained using this partition are therefore described as internal random-split results and are not interpreted as evidence of route-independent or external generalization.

Table 3. Original random image-level partition used in the primary experiments and the multiple-seed variability analysis.

Subset	Number of Images	Percentage
Training	1400	70%
Validation	400	20%
Test	200	10%
Total	2000	100%

To assess performance under route-disjoint data partitions, two complementary route assignments were evaluated. Images from R01 and R02 were used for training in both assignments, comprising 1394 images. In Split A, the 324 images from R03 were used for validation and checkpoint selection, and the 282 images from R04 were reserved for final testing. In Split B, the roles of R03 and R04 were exchanged: R04 was used for validation and checkpoint selection, and R03 was reserved for final testing.

A separate RFP-YOLO26 model was trained for each split and random seed. Random seeds 0 and 1 were evaluated for both held-out test routes. The principal seed-0 comparison between YOLOv26n and RFP-YOLO26 was conducted under Split A. In each experiment, the corresponding held-out test route was excluded from model training, hyperparameter adjustment, early stopping, and checkpoint selection.

A documented duplicate and near-duplicate audit was conducted across the three route subsets. No byte-identical or pixel-identical cross-subset image pairs were detected. Perceptual hashing produced 158 candidate pairs for manual inspection. These candidates exhibited similar pavement textures, road boundaries, or acquisition viewpoints but represented distinct physical scenes. None was confirmed as a cross-subset duplicate or near-duplicate pair.

The individual route-disjoint results, descriptive route-level summaries, and complete audit details are provided in Supplementary Table S3 and Supplementary Note S1.

4.4. Experimental Configuration

Training was conducted on a workstation equipped with an Intel Core i7-14700 CPU, an NVIDIA GeForce RTX 4060 Ti GPU with 16 GB of video memory, and 64 GB of system memory. The software environment consisted of Microsoft Windows, Python 3.10.19, PyTorch 2.0.1 with CUDA 11.8, and Ultralytics 8.4.27.

YOLOv26n was used as the baseline detector. RFP-YOLO26 was implemented in the same framework using a custom YAML model configuration and the SPDConv, C3k2_Faster_EMA, SimAM, and fixed single-cycle feedback modules described in Section 3. The final configuration was stored in `YOLOv26n-RFP-SimAM.yaml`. Models were instantiated from their YAML architecture configurations, and no external `.pt` checkpoint was explicitly loaded in the reported training scripts.

For the controlled alternative-neck experiment, a BiFPN-style control, referred to throughout the manuscript as the BiFPN-control, was constructed by replacing the fixed single-cycle feedback neck of RFP-YOLO26 with a single weighted bidirectional fusion neck. The backbone, SPDConv layers, C3k2_Faster_EMA modules, SimAM modules, detection head, input resolution, and other principal detector components were kept unchanged. The BiFPN-control did not include the proposed second top-down feedback-refinement path.

At each BiFPN fusion node, nonnegative learnable weights were obtained by applying a rectified linear function and were normalized by their sum plus an epsilon of 10^{-4}. The BiFPN-control was instantiated directly from its architecture definition without explicitly loading an external `.pt` checkpoint.

The principal training settings are listed in Table 4. Within each experimental protocol, the compared models used the same input resolution, optimization settings, augmentation settings, checkpoint-selection procedure, and evaluation settings.

Table 4. Common training settings used in the reported experiments. Dataset partitions and random seeds are specified separately for the corresponding evaluation protocols.

Setting	Value
Framework	Ultralytics 8.4.27
Input size	640 × 640
Maximum epochs	400
Training batch size	48
Optimizer	SGD
Initial learning rate	0.01
Final learning-rate factor	0.01
Momentum	0.937
Weight decay	0.0005
Cosine learning-rate schedule	Disabled
Warm-up epochs	3
Warm-up momentum	0.8
Warm-up bias learning rate	0.1
Bounding-box regression loss weight	7.5
Classification loss weight	0.5
External checkpoint explicitly loaded	No
Random seed	0 for the primary benchmark, BiFPN-control, component-integration, and feedback-cycle experiments; 0–4 for the original-split five-seed analysis; 0 and 1 for each route-disjoint split
Deterministic execution	True for every run
Automatic mixed precision	True
Early-stopping patience	100 epochs

The primary benchmark, controlled BiFPN-style neck comparison, progressive component-integration experiments, and feedback-cycle experiments were conducted using random seed 0. Under the route-disjoint protocol, RFP-YOLO26 was trained independently using random seeds 0 and 1 for each of the two route assignments. To assess between-run variability under the original random image-level partition, YOLOv26n and RFP-YOLO26 were additionally trained independently using five random seeds (0–4).

Each run was initialized independently, and no checkpoint from a preceding run was used to initialize a subsequent run. Apart from the specified dataset partition and random seed, the training, checkpoint-selection, and evaluation settings were kept unchanged within each protocol.

The BiFPN-control and the seed-0 RFP-YOLO26 model used the same original random image-level partition, optimization settings, augmentation settings, checkpoint-selection procedure, test subset, evaluation arguments, and numerical-precision settings. Apart from the explicitly specified dataset partition and random seed, all other training and evaluation settings were kept unchanged. The complete training, augmentation, and evaluation settings are provided in Supplementary Table S1. The complete per-run results and statistical summaries of the five-seed analysis are reported in Supplementary Table S2.

4.5. Data Augmentation

Mosaic augmentation was enabled with a probability of 1.0 throughout training (`close_mosaic=0`). The HSV gains for hue, saturation, and value were 0.015, 0.7, and 0.4,

respectively. Translation and scaling were set to 0.1 and 0.5, and horizontal flipping was applied with a probability of 0.5.

Vertical flipping, rotation, shearing, perspective transformation, MixUp, and copy-paste augmentation were disabled. The complete augmentation configuration is provided in Supplementary Table S1.

4.6. Checkpoint Selection, Evaluation Settings, and Statistical Analysis

Validation was performed after each training epoch. For every independent run, the checkpoint selected automatically according to validation fitness was stored as `best.pt`. All test results were obtained using the corresponding `best.pt` checkpoint rather than the final-epoch checkpoint.

Under the original random image-level partition, the selected checkpoints were evaluated on the same 200-image test subset. Under the route-disjoint protocol, a separate checkpoint was selected for each split and random seed. In Split A, checkpoint selection was based exclusively on R03 and final testing was conducted on R04. In Split B, checkpoint selection was based exclusively on R04 and final testing was conducted on R03. The held-out test route was not used for training, hyperparameter adjustment, early stopping, or checkpoint selection in the corresponding experiment.

For both protocols, evaluation used an input size of 640×640, a batch size of 48, and the framework arguments `conf=0.001`, `iou=0.7`, `max_det=300`, `half=False`, `augment=False`, and `agnostic_nms=False`. Precision, recall, AP, and mAP were returned directly by the Ultralytics 8.4.27 validation pipeline using FP32 inference without test-time augmentation.

To assess between-run variability under the original random image-level partition, YOLOv26n and RFP-YOLO26 were independently trained using five random seeds (0–4). All other training, checkpoint-selection, and evaluation settings were kept unchanged. For each detection metric, the arithmetic mean, sample standard deviation, and two-sided 95% confidence interval for the mean were calculated across the five independent runs. The confidence interval was computed as

$$\bar{x} \pm t_{0.975,4} \frac{s}{\sqrt{5}}, \tag{25}$$

where $\bar{x}$ is the five-run mean, s is the sample standard deviation, and $t_{0.975,4} = 2.776$. Complete per-run results and summary statistics are provided in Supplementary Table S2.

The cross-generation benchmark, controlled BiFPN-style neck comparison, progressive component-integration experiments, and feedback-cycle experiments were conducted once using random seed 0 and are reported as descriptive point-estimate comparisons.

For the extended route-disjoint evaluation, the individual results for random seeds 0 and 1 were reported for each held-out test route. Because two runs were evaluated per route, the arithmetic mean and observed minimum–maximum range are reported descriptively in Supplementary Table S3.

No formal statistical-significance claim is made for the seed-0 comparisons or the two-run route-level summaries.

4.7. Complexity Profiling

Parameter counts and GFLOPs were obtained using the same profiling procedure for all models, including the BiFPN-control, with an input resolution of 640×640 pixels and a profiling batch size of 1.

The reported GFLOPs describe model-level computation and do not include image acquisition, data transfer, preprocessing, post-processing, visualization, communication,

or vehicle-control latency. Embedded execution was therefore evaluated separately on the Jetson Orin Nano.

4.8. On-Vehicle Runtime Configuration

The on-vehicle runtime was implemented in Python using the Ultralytics detection interface and OpenCV. The vehicle-mounted GMSL camera provided a 1920 × 1080 stream at 30 FPS. A GStreamer pipeline with `drop=true`, `sync=false`, and `max-buffers=1` was used to limit the accumulation of outdated frames.

The detector received images at 640 × 640 pixels and used FP16 inference on the Jetson Orin Nano. One zero-valued input image was processed before vehicle operation as a model warm-up. Inference was restricted to class 0.class. The runtime confidence and IoU parameters were set to 0.35 and 0.45, respectively.

For each detected obstacle cluster, the runtime calculated the centroid of the enclosing bounding box. A precomputed homography was used to transform the image coordinate to a vehicle-relative coordinate, which was then transmitted through the vehicle communication interface.

The minimum interval between obstacle-coordinate messages was 0.2 s, corresponding to a maximum nominal output frequency of 5 Hz. During the reported on-vehicle experiments, the street-sweeping vehicle was operated at approximately 1 m/s under the reported low-speed experimental configuration.

Supporting photographs of the camera installation, data-acquisition equipment, embedded-computing and camera-interface hardware, and on-vehicle operation interface are provided in Supplementary Figure S2. These photographs document the reported experimental setup and are not used as a quantitative performance, reliability, or safety evaluation.

4.9. Assessment Criteria

Detection performance was evaluated using precision, recall, AP@0.5, and AP@0.5:0.95 under the axis-aligned bounding-box protocol implemented by Ultralytics 8.4.27. AP@0.5 evaluates average precision at an IoU threshold of 0.5, whereas AP@0.5:0.95 averages AP over IoU thresholds from 0.50 to 0.95 in increments of 0.05 following the COCO-style convention [30].

Because USLO contains only the class, the class-specific AP is numerically equal to the reported single-class mAP under the corresponding IoU protocol. Model complexity was described using parameter count and GFLOPs.

Embedded execution was described using measured throughput in frames per second. The reciprocal of FPS was used only as a nominal average interval between successive model outputs. It was not treated as a measured end-to-end latency distribution or as a quantitative perception-to-control safety endpoint.

For the explicit false-positive and false-negative analysis, the seed-0 RFP-YOLO26 checkpoint from route-disjoint Split A was evaluated on the held-out R04 route. The operating confidence threshold was set to 0.44 based on the maximum of the F1–confidence curve on the R03 validation route and was then applied unchanged to R04.

Predictions were processed in descending confidence order and were matched one-to-one to ground-truth boxes. A match with IoU $\geq$ 0.5 was counted as a true positive. Unmatched predictions were counted as false positives, and unmatched ground-truth instances were counted as false negatives. Each false-positive and false-negative case was visually reviewed and assigned to one mutually exclusive primary category.

Apparent object width was defined as the shorter side of each ground-truth bounding box after proportional scaling to the 640-pixel model input. For descriptive stratification,

the 33rd and 67th percentiles of the R04 apparent-width distribution were used as group boundaries, corresponding to 10.88 and 30.67 pixels. These groups were used only to summarize recall by apparent width and did not affect training, checkpoint selection, confidence-threshold selection, or detection matching.

4.10. Reproducibility Materials

The supplementary reproducibility package contains the final RFP-YOLO26 architecture configuration, the zero-, one-, and two-cycle feedback configurations, the training and evaluation scripts, the modified Ultralytics source files required to define and register the custom modules, the trained checkpoint of the final RFP-YOLO26 model, a representative annotated data subset, supplementary documents, and a README file describing installation and execution.

Supplementary Table S1 provides the complete training, augmentation, checkpoint-selection, and evaluation settings.

The accompanying supplementary PDF contains Supplementary Table S2 with the individual five-seed results and statistical summaries under the original random image-level partition; Supplementary Table S3 and Supplementary Note S1 with the extended route-disjoint evaluation and cross-subset duplicate and near-duplicate audit; Supplementary Table S4 with the BiFPN-style alternative-neck comparison; Supplementary Table S5 with the fixed-operating-point false-positive and false-negative analysis; and Supplementary Table S6 with the apparent-width-stratified recall analysis.

Supplementary Figures S1 and S2 provide the evaluation curves and experimental-setup photographs, respectively.

The model configuration files are listed below:

- `YOLOv26n-RFP-SimAM.yaml`;
- `RFP-YOLO26-Feedback0.yaml`;
- `RFP-YOLO26-Feedback1.yaml`;
- `RFP-YOLO26-Feedback2.yaml`.

Supplementary Data S1 contains a representative annotated subset of the USLO dataset, including image files and corresponding YOLO-format label files. This subset is provided for inspection of the annotation format and representative scene content; it is not an independent test set and was not used for additional metric calculation.

The final RFP-YOLO26 model was instantiated directly from its YAML architecture configuration, and no external `.pt` checkpoint was loaded for training initialization. The validation split was used during training for checkpoint selection, whereas the primary random-split test metrics were generated separately by `evaluate.py` on the 200-image test split using the selected `RFP-YOLO26_best.pt` checkpoint. The route-disjoint results were generated on the corresponding held-out test routes using the split-specific checkpoints described in Section 4.3.

These materials are provided to support inspection of the implemented architecture, annotation format, and reported experimental procedure. The complete USLO dataset and project-specific vehicle-control software are not released for unrestricted external distribution because of confidentiality and intellectual-property restrictions.

5. Experimental Results and Discussion

5.1. Descriptive Comparison with Compact YOLO Detectors

RFP-YOLO26 was compared with YOLOv8n, YOLOv9n, YOLOv10n, YOLOv11n, and YOLOv26n using the original random image-level partition. All models used the common training, checkpoint-selection, and evaluation settings described in Sections 4.4 and 4.6.

Each entry in Table 5 corresponds to a single run using random seed 0. The table is therefore presented as a descriptive engineering benchmark rather than as an architecture-optimized or repeated-run comparison.

Table 5. Seed-0 descriptive performance and computational-complexity comparison on the original internal random-split test set. The five-seed analysis of YOLOv26n and RFP-YOLO26 is reported in Supplementary Table S2.

Model	mAP@0.5 (%)	mAP@0.5:0.95 (%)	Precision (%)	Recall (%)	GFLOPs	Params (M)
YOLOv8n	94.9	63.8	96.9	90.7	6.8	2.68
YOLOv9n	95.1	63.4	96.4	91.9	6.6	1.71
YOLOv10n	95.2	62.8	97.4	90.4	6.5	2.27
YOLOv11n	94.9	62.1	97.9	91.8	6.3	2.58
YOLOv26n	95.0	62.8	96.4	91.3	5.2	2.38
RFP-YOLO26	97.9	65.0	98.1	95.6	9.9	4.21

In the seed-0 comparison, RFP-YOLO26 exceeded YOLOv26n by 2.9 percentage points in mAP@0.5, 2.2 percentage points in mAP@0.5:0.95, 1.7 percentage points in precision, and 4.3 percentage points in recall.

To assess between-run variability, YOLOv26n and RFP-YOLO26 were additionally trained using five independent random seeds under the same random-split protocol. RFP-YOLO26 retained higher mean values for all four metrics, with mean differences of 2.66, 2.00, 1.42, and 4.02 percentage points, respectively. The individual results, sample standard deviations, and 95% confidence intervals are reported in Supplementary Table S2. No formal paired statistical-significance claim is made.

Under route-disjoint Split A using random seed 0, YOLOv26n achieved 94.3% mAP@0.5, 61.6% mAP@0.5:0.95, 95.7% precision, and 89.4% recall on the held-out R04 route. RFP-YOLO26 achieved corresponding values of 97.4%, 64.0%, 96.8%, and 95.1%. The direction of the principal model comparison was therefore retained under this route assignment.

RFP-YOLO26 was further evaluated using random seeds 0 and 1 under both complementary route-disjoint splits. For R04 as the held-out test route, the route-level mean values were 97.3% mAP@0.5, 63.7% mAP@0.5:0.95, 96.4% precision, and 94.8% recall. For R03 as the held-out test route, the corresponding mean values were 97.6%, 64.6%, 97.1%, and 95.3%. The individual results and observed ranges are provided in Supplementary Table S3.

Because the compared YOLO generations differ in their backbones, feature-fusion structures, detection heads, and other implementation details, Table 5 should not be interpreted as a controlled comparison of feature-pyramid necks. It characterizes only the performance and computational cost of the evaluated complete detectors under the common protocol.

The improved detection performance was accompanied by increased computational cost. Compared with YOLOv26n, RFP-YOLO26 adds 1.83 M parameters and 4.7 GFLOPs. It is therefore interpreted as an accuracy–complexity trade-off rather than as a lower-compute replacement for YOLOv26n. In the controlled alternative-neck comparison, the BiFPN-control contained 4.13 M parameters and required 9.27 GFLOPs, compared with 4.21 M parameters and 9.90 GFLOPs for RFP-YOLO26. Under the common seed-0 protocol, the BiFPN-control achieved 96.9% mAP@0.5, 63.9% mAP@0.5:0.95, 97.2% precision, and 93.5% recall, whereas RFP-YOLO26 achieved 97.9%, 65.0%, 98.1%, and 95.6%, respectively. These results are reported as descriptive point estimates. The complete comparison is provided in Supplementary Table S4.

5.2. Training-Curve Interpretation

The training and validation curves of the seed-0 RFP-YOLO26 run under the original random-split protocol are shown in Figure 7. These curves describe the optimization trajectory, validation behavior, and checkpoint-selection process of this individual run.

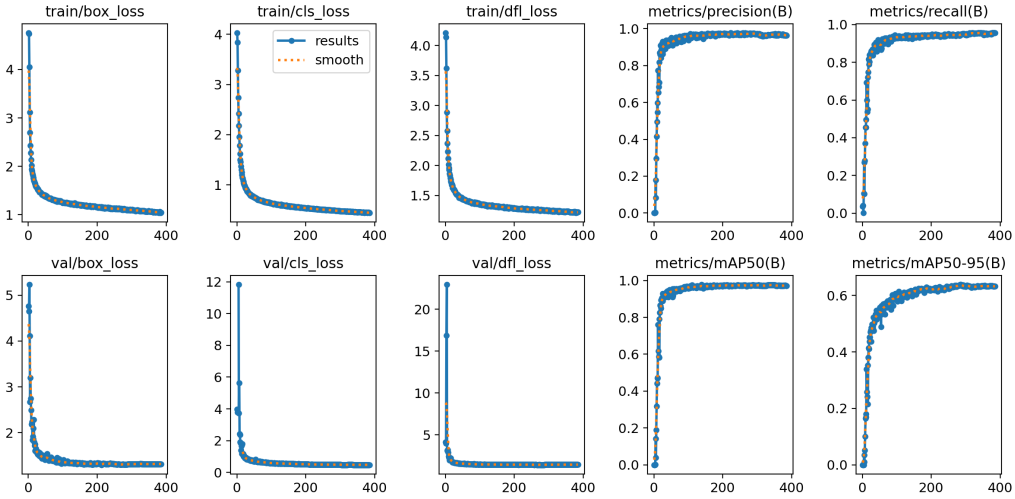


Figure 7. Training and validation curves of the seed-0 RFP-YOLO26 run under the original random-split protocol. The figure represents one optimization trajectory; the five-seed analysis of the final test metrics is reported in Supplementary Table S2.

The displayed curves indicate that the seed-0 run reached a numerically stable region before training termination. Nevertheless, Figure 7 represents only one optimization trajectory and should not be interpreted as a visual summary of all five runs. Between-run variability in the final test metrics is reported in Supplementary Table S2; complete training curves for the remaining seeds are not displayed.

5.3. Progressive Component Integration Study

To document the performance trajectory followed during model development, a progressive component-integration study was conducted. Starting from the YOLOv26n baseline, the fixed single-cycle feedback feature pyramid, SPDConv, C3k2_Faster_EMA, and SimAM were incorporated sequentially according to the predetermined model-development order. The resulting cumulative configurations are reported in Table 6.

Each configuration contains all components included in the preceding row. Accordingly, differences between adjacent rows are interpreted as changes between cumulative configurations rather than as independent effects of individual components.

Table 6. Descriptive performance trajectory of progressively integrated model configurations. Each row represents a cumulative configuration and should not be interpreted as an independent estimate of the effect of the newly introduced component, module interactions, or module synergy.

Model Variant	Feedback Pyramid	SPDConv	C3k2_Faster_EMA	SimAM	mAP@0.5 (%)	mAP@0.5:0.95 (%)	Params (M)
YOLOv26n	–	–	–	–	95.0	62.8	2.38
+Feedback Pyramid	✓	–	–	–	96.9	63.6	3.34
+Feedback Pyramid + SPDConv	✓	✓	–	–	97.4	64.1	3.81
+Feedback Pyramid + SPDConv + C3k2_Faster_EMA	✓	✓	✓	–	97.6	63.8	4.21
RFP-YOLO26	✓	✓	✓	✓	97.9	65.0	4.21

The successive cumulative configurations produced mAP@0.5 point estimates of 95.0%, 96.9%, 97.4%, 97.6%, and 97.9%, respectively. The corresponding mAP@0.5:0.95 point estimates were 62.8%, 63.6%, 64.1%, 63.8%, and 65.0%. These values describe the performance trajectory associated with the selected model-development order.

Among the cumulative configurations evaluated in Table 6, the complete RFP-YOLO26 system produced the highest mAP point estimates. Relative to YOLOv26n, the complete system increased mAP@0.5 and mAP@0.5:0.95 by 2.9 and 2.2 percentage points, respectively, while increasing the parameter count from 2.38 M to 4.21 M. These results therefore describe the observed accuracy–complexity trade-off of the complete integrated system under the reported experimental protocol.

Accordingly, Table 6 supports conclusions about the evaluated cumulative configurations and the complete RFP-YOLO26 system.

The progressive component-integration study and the feedback-depth comparison address different questions: the former reports cumulative model-development configurations, whereas the latter compares feedback-cycle depth within the same general detector design.

5.4. Descriptive Accuracy–Complexity Comparison of Feedback-Cycle Configurations

To descriptively compare the accuracy–complexity characteristics of different feedback-depth configurations, models with zero, one, and two feedback cycles were evaluated under the same dataset partition, training settings, checkpoint-selection rule, and evaluation protocol.

Feedback-0 contains only the initial top-down feature-fusion stage and does not include a bottom-up feedback pass or a second top-down refinement stage. Feedback-1 corresponds to the proposed RFP-YOLO26 topology, consisting of an initial top-down fusion pass, one bottom-up feedback pass, and one second top-down refinement pass. Feedback-2 extends Feedback-1 by adding a second independently instantiated bottom-up feedback and top-down refinement cycle.

The stage-specific modules are independently instantiated, and weights are not shared across the feedback cycles. Because each additional cycle introduces extra downsampling and feature-fusion modules, the parameter counts and GFLOPs are not matched among the three configurations. This experiment therefore evaluates the practical accuracy–complexity trajectory associated with feedback depth rather than a strictly compute-matched comparison.

As shown in Table 7, Feedback-1 produced the highest point estimates for all four reported detection metrics. Compared with Feedback-0, Feedback-1 increased mAP@0.5 from 97.2% to 97.9% and mAP@0.5:0.95 from 63.3% to 65.0%, corresponding to increases of 0.7 and 1.7 percentage points, respectively. Precision increased from 97.0% to 98.1%, while recall increased from 94.4% to 95.6%.

Table 7. Comparison of fixed feature-feedback topologies with zero, one, and two feedback cycles on the same 200-image internal test set.

Configuration	Params (M)	GFLOPs	Precision (%)	Recall (%)	mAP@0.5 (%)	mAP@0.5:0.95 (%)
Feedback-0	3.01	7.8	97.0	94.4	97.2	63.3
Feedback-1 (RFP-YOLO26)	4.21	9.9	98.1	95.6	97.9	65.0
Feedback-2	5.40	11.9	97.5	94.8	97.3	63.9

Extending the topology from Feedback-1 to Feedback-2 increased the parameter count from 4.21 M to 5.40 M and GFLOPs from 9.9 to 11.9. However, mAP@0.5 decreased from 97.9% to 97.3%, and mAP@0.5:0.95 decreased from 65.0% to 63.9%. Precision and recall also decreased from 98.1% and 95.6% to 97.5% and 94.8%, respectively.

Among the three evaluated configurations, Feedback-1 produced the highest single-run detection point estimates under the reported protocol. Because parameter count and GFLOPs increased with feedback depth, Table 7 is interpreted as a within-family accuracy–complexity comparison of the three implemented configurations.

An additional approximately compute-matched BiFPN-style control was evaluated under the same seed-0 random-split protocol. The control did not contain the proposed second top-down feedback-refinement path. Its complete results are reported in Supplementary Table S4.

Table 7 is therefore interpreted as a within-family comparison of the three implemented feedback-depth configurations, while Supplementary Table S4 provides the complementary alternative-neck control.

5.5. Selected Qualitative Detection Examples

Figure 8 presents selected detection outputs produced by YOLOv26n and RFP-YOLO26 in road-scene examples containing slender obstacles. The images illustrate that the two models can generate different sets of bounding-box predictions under the displayed conditions.

The examples are not accompanied by a separately defined field-evaluation protocol, independent ground-truth review, event-level denominators, or confidence intervals. Consequently, the number of displayed prediction boxes is not treated as an independent measure of accuracy. A larger number of boxes could represent additional true detections, additional false positives, or a combination of both.

The figures also do not provide segmentation masks, object centerlines, or continuity-sensitive annotations. They therefore cannot establish that one model produces more continuous object representations or less object fragmentation. Quantitative comparison between the models is based on the annotated internal test set and the metrics reported in Table 5, rather than on selected images.

The selected examples show that RFP-YOLO26 can produce bounding-box predictions in scenes containing pavement textures, shadows, illumination changes, and partial visual obstruction. However, these figures are illustrative and do not establish robustness across unobserved environments.

Separately from these illustrative examples, the fixed-operating-point analysis on the held-out R04 route produced 405 true positives, 16 false positives, and 23 false negatives among 428 annotated instances. The corresponding fixed-operating-point precision and recall were 96.2% and 94.6%.

Pavement cracks or expansion joints accounted for 9 of the 16 false positives, while narrow or small apparent size was assigned as the primary category for 12 of the 23 false negatives. Recall was 91.0% for the narrow apparent-width group, 96.1% for the medium group, and 97.1% for the wide group. Complete error-category and apparent-width results are provided in Supplementary Tables S5 and S6.

5.6. Embedded Deployment Evaluation

RFP-YOLO26 and YOLOv26n were deployed on the Jetson Orin Nano to measure platform-specific inference throughput under the same reported deployment configuration. The results are summarized in Table 8.

YOLOv26n achieved approximately 15.0 FPS, whereas RFP-YOLO26 achieved approximately 8.2 FPS. The reduction in throughput is consistent with the increase in parameter count, GFLOPs, and feature-fusion stages. Therefore, the proposed detector represents an accuracy–complexity trade-off rather than a lower-compute replacement for YOLOv26n.



(**a**) YOLOv26n predictions



(**b**) RFP-YOLO26 predictions

Figure 8. Selected qualitative bounding-box predictions produced by YOLOv26n and RFP-YOLO26. The displayed images are provided for visual inspection and are not used as a separately sampled quantitative field-test benchmark.

Table 8. Model complexity and measured inference throughput on the Jetson Orin Nano under the reported deployment configuration.

Model	Params (M)	GFLOPs	Throughput (FPS)
YOLOv26n	2.38	5.2	15.0
RFP-YOLO26	4.21	9.9	8.2

The on-vehicle runtime used FP16 inference, model warm-up, an input size of 640 × 640 pixels, and a low-buffer camera pipeline. During the reported experiments, the vehicle was operated at approximately 1 m/s under the reported low-speed experimental configuration. At 8.2 FPS, the nominal interval between successive model outputs was approximately 0.122 s, corresponding to approximately 0.122 m of vehicle travel at 1 m/s.

The runtime limited obstacle-coordinate messages to a minimum interval of 0.2 s, corresponding to a maximum nominal output frequency of 5 Hz. At 1 m/s, the vehicle traveled approximately 0.20 m during this communication interval. The measured model-output rate therefore exceeded the configured maximum nominal coordinate-output frequency under this specific setup.

The reported throughput and message intervals characterize only the specified Jetson Orin Nano and low-speed operating configuration and are not interpreted as an end-to-end vehicle-safety or operational-reliability evaluation.

5.7. Qualitative On-Vehicle Examples

Figure 9 presents selected images obtained while the model was executed on the autonomous street-sweeping vehicle. These examples show the vehicle platform, camera configuration, and bounding-box outputs generated in several displayed road scenes.

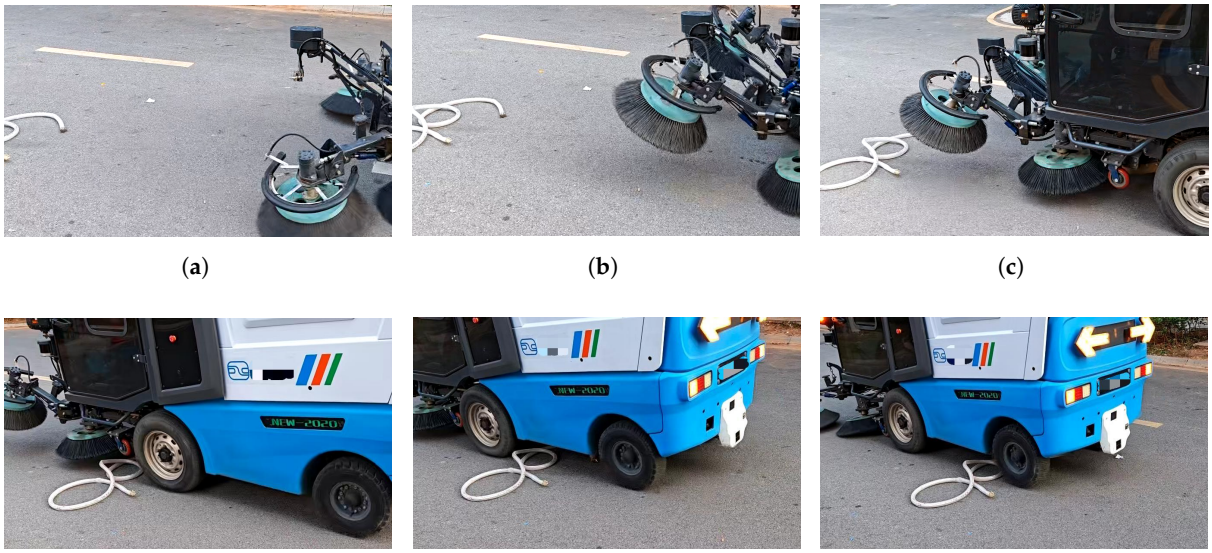


Figure 9. Selected qualitative sequence showing the vehicle response after detection of a slender strip-like obstacle: (**a**) detection of the obstacle; (**b**) initiation of sweeping-brush lifting; (**c**) sweeping brush in the raised position; (**d**) vehicle passing the obstacle with the sweeping brush raised; (**e**) lowering of the sweeping brush after passing the obstacle; and (**f**) resumption of normal vehicle operation. The sequence illustrates the implemented perception–response process and is not used as a quantitative safety or reliability benchmark.

The displayed cases document integration with the vehicle-mounted camera and embedded computing platform and illustrate example prediction outputs during operation. They are interpreted as qualitative examples only and are not used as a quantitative field, reliability, or safety benchmark.

5.8. Discussion

Under the current experimental protocol, RFP-YOLO26 produced higher descriptive point estimates than YOLOv26n. The differences were 2.9 percentage points in mAP@0.5, 2.2 percentage points in mAP@0.5:0.95, 1.7 percentage points in precision, and 4.3 percentage points in recall. The differences reported in Table 5 are seed-0 descriptive point estimates. A supplementary five-seed analysis retained the same model ordering for all four detection metrics, with complete results reported in Supplementary Table S2. However, no formal paired statistical-significance claim is made.

A possible explanation is that slender obstacles occupy only a narrow transverse region in an image, and their fine-scale responses may be weakened during repeated downsampling. The proposed neck performs an initial top-down fusion, one bottom-up feedback pass, and a second top-down refinement pass. This fixed sequence provides an additional opportunity to reuse fine spatial evidence after cross-scale aggregation. SPD-Conv, C3k2_Faster_EMA, and SimAM further modify downsampling, feature extraction, and output-scale weighting. This interpretation represents an architectural rationale rather than an independently verified causal mechanism.

The progressive component-integration study should be interpreted cautiously because the modules were added cumulatively. It therefore describes the performance trajectory of successive configurations but does not isolate independent component effects or formally establish module synergy.

The feedback-depth comparison provides a descriptive within-family accuracy–complexity comparison of the three implemented configurations. Feedback-1 achieved the highest point estimates among the zero-, one-, and two-cycle configurations. Com-

pared with Feedback-0, it increased precision from 97.0% to 98.1%, recall from 94.4% to 95.6%, mAP@0.5 from 97.2% to 97.9%, and mAP@0.5:0.95 from 63.3% to 65.0%. Feedback-2 increased the parameter count from 4.21 M to 5.40 M and GFLOPs from 9.9 to 11.9, but reduced the corresponding metrics to 97.5%, 94.8%, 97.3%, and 63.9%. These results support Feedback-1 as the most favorable empirical accuracy–complexity operating point among the three evaluated configurations, but not as a theoretically or universally optimal feedback depth.

The proposed detector is intended for low-speed autonomous street-sweeping operation on conventional paved road surfaces, including cement-paved surfaces and similar regular road environments. A supplementary fixed-operating-point analysis quantified the principal observed false-positive and false-negative categories on the held-out R04 route and stratified recall by apparent object width. Recall was 91.0% for the narrow group, compared with 96.1% for the medium group and 97.1% for the wide group.

An elongated pavement crack also produced a false positive in one selected on-vehicle example. This field observation is illustrative and is reported separately from the fixed-operating-point R04 error analysis.

The higher detection point estimates were accompanied by increased computational cost. RFP-YOLO26 contains 4.21 M parameters and requires 9.9 GFLOPs, compared with 2.38 M parameters and 5.2 GFLOPs for YOLOv26n. Its measured Jetson Orin Nano throughput was approximately 8.2 FPS, compared with approximately 15.0 FPS for the baseline. During the on-vehicle experiments, the vehicle was operated at approximately 1 m/s. At 8.2 FPS, the nominal interval between model outputs is approximately 0.122 s, corresponding to approximately 0.122 m of vehicle travel. The runtime transmitted obstacle coordinates at a maximum nominal frequency of 5 Hz, corresponding to approximately 0.20 m between messages. These values indicate compatibility with the current low-speed configuration but do not establish a universal safe frame rate or quantitative safety certification.

The primary benchmark continues to use the original random image-level partition. The route-disjoint evaluation was expanded to two complementary held-out test routes, with two independently trained random-seed runs for each route. The individual results and observed ranges are reported in Supplementary Table S3, and the cross-subset audit identified no exact or manually confirmed near-duplicate pairs. These results provide an additional route-separated assessment within the present dataset and are interpreted descriptively within the reported protocol. These results provide an additional route-separated assessment within the present dataset and are interpreted descriptively within the reported protocol.

The progressive component-integration and feedback-cycle experiments remain seed-0 descriptive evaluations. The overall findings therefore characterize the observed accuracy–complexity behavior of the tested complete configurations rather than formal statistical or operational claims.

A controlled BiFPN-style alternative-neck comparison was conducted under the original random image-level partition. The BiFPN-control used a closely aligned computational budget and the same principal detector framework and experimental protocol as RFP-YOLO26. RFP-YOLO26 produced higher seed-0 point estimates for the four reported detection metrics, while the BiFPN-control used modestly fewer parameters and GFLOPs. The comparison is interpreted descriptively at the level of the evaluated complete neck configurations.

6. Conclusions

This study presented RFP-YOLO26 for the single-class axis-aligned bounding-box detection of slender obstacles in low-speed autonomous street-sweeping environments. The detector combines established SPDConv, C3k2_Faster_EMA, and SimAM modules with

a fixed single-cycle feedback feature pyramid consisting of an initial top-down fusion pass, one bottom-up feedback pass, and a second top-down refinement pass. The contribution lies in the task-oriented implementation and evaluation of this architecture rather than in the individual invention of the incorporated operators.

The progressive component-integration experiment describes the performance trajectory of the tested cumulative configurations. Its results are therefore interpreted at the level of the complete integrated detector rather than as independent module effects.

On the internal random-split test set, RFP-YOLO26 achieved 97.9% mAP@0.5, 65.0% mAP@0.5:0.95, 98.1% precision, and 95.6% recall. Compared with YOLOv26n, these values increased by 2.9, 2.2, 1.7, and 4.3 percentage points, respectively. This improvement was accompanied by an increase in parameter count from 2.38 M to 4.21 M and in GFLOPs from 5.2 to 9.9. The measured Jetson Orin Nano throughput decreased from approximately 15.0 FPS to 8.2 FPS. RFP-YOLO26 should therefore be interpreted as an accuracy–complexity trade-off rather than as a lower-compute replacement for YOLOv26n.

The feedback-depth experiment showed that the single-cycle configuration achieved the highest detection point estimates among the evaluated zero-, one-, and two-cycle variants. Adding a second feedback cycle increased the parameter count and GFLOPs but did not improve the reported detection metrics. Table 7 is therefore interpreted as a within-family comparison of the three implemented feedback-depth configurations.

In the approximately compute-matched alternative-neck comparison, the BiFPN-control contained 4.13 M parameters and required 9.27 GFLOPs, whereas RFP-YOLO26 contained 4.21 M parameters and required 9.90 GFLOPs. Under the common seed-0 protocol, RFP-YOLO26 produced higher descriptive point estimates for the four reported detection metrics.

The detector achieved approximately 8.2 FPS on the Jetson Orin Nano during the reported on-vehicle experiments at an operating speed of approximately 1 m/s on conventional paved roads. This result indicates embedded execution feasibility under the reported hardware and low-speed configuration, but it does not constitute a quantitative vehicle-safety or operational-reliability evaluation.

The cross-generation benchmark, progressive component-integration experiment, feedback-cycle comparison, and BiFPN-style control remain seed-0 descriptive analyses. YOLOv26n and RFP-YOLO26 were additionally evaluated across five random seeds under the original random-split protocol.

The extended route-disjoint evaluation used two complementary held-out test routes and two random seeds per route. The fixed-operating-point analysis on R04 further reported explicit true-positive, false-positive, and false-negative counts and apparent-width-stratified recall. Complete results are provided in Supplementary Tables S2–S6.

These additional analyses are interpreted within the evaluated dataset, partitioning protocols, model configurations, and reported hardware setup. The conclusions remain restricted to the observed performance and accuracy–complexity characteristics of the complete integrated RFP-YOLO26 system.

Supplementary Materials: The following supporting information can be downloaded at: https://www.mdpi.com/article/10.3390/app16167991/s1. Supplementary Table S1: Complete training, data-augmentation, checkpoint-selection, and evaluation settings; Supplementary Figure S1: Precision–recall and F1–confidence curves; Supplementary Figure S2: Experimental-setup photographs documenting the camera installation, data-acquisition equipment, embedded-computing and camera-interface hardware, and on-vehicle operation interface, contains the complete training, data-augmentation, checkpoint-selection, and evaluation settings. The accompanying supplementary PDF contains Supplementary Table S2, reporting the individual five-seed results and statistical summaries under the original random image-level partition; Supplementary Table S3 and Supplementary Note S1, reporting the extended

route-disjoint evaluation and cross-subset duplicate and near-duplicate audit; Supplementary Table S4, reporting the BiFPN-style alternative-neck comparison; Supplementary Table S5, reporting the fixed-operating-point false-positive and false-negative analysis; and Supplementary Table S6, reporting the apparent-width-stratified recall analysis. Supplementary Figures S1 and S2 provide the evaluation curves and experimental-setup photographs, respectively.

Author Contributions: Conceptualization, Z.C. (Zhongwen Chen) and Q.Z.; methodology, Z.C. (Zhongwen Chen); software, Z.C. (Zhongwen Chen); validation, Z.C. (Zhongwen Chen); formal analysis, Z.C. (Zhongwen Chen); investigation, Q.Z.; resources, Z.C. (Zhongwen Chen), Z.C. (Zihua Chen) and H.Y.; data curation, Z.C. (Zhongwen Chen) and H.Y.; writing—original draft preparation, Z.C. (Zhongwen Chen); writing—review and editing, Z.C. (Zhongwen Chen) and Q.Z.; visualization, Z.C. (Zhongwen Chen); supervision, Q.Z., Z.C. (Zihua Chen), Y.Z., H.Y. and Q.W.; project administration, Q.Z., Z.C. (Zihua Chen), Y.Z., H.Y. and Q.W. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no direct financial funding. Hunan Niu En Chi New Energy Vehicle provided non-financial, in-kind support consisting of an autonomous sweeping vehicle, camera equipment, testing facilities, road-scene data, and field engineering assistance.

Institutional Review Board Statement: Not applicable. This study did not involve human participants, animals, clinical procedures, or the collection of identifiable personal data.

Informed Consent Statement: Not applicable.

Data Availability Statement: A representative annotated subset of the USLO dataset, including image files and the corresponding YOLO-format labels, is provided as Supplementary Data S1. This subset is intended to support inspection of the annotation format and representative scene content and is not an independent evaluation dataset. The complete USLO dataset is not publicly available because it constitutes project-specific research data that are subject to access and external-distribution restrictions. Access to the complete dataset is limited to the authors and authorized project personnel. Further inquiries regarding data availability may be directed to the corresponding author.

Conflicts of Interest: Author Heng Yang is employed as Technical Director of Hunan Niu En Chi New Energy Vehicle. The company provided non-financial, in-kind support consisting of an autonomous sweeping vehicle, camera equipment, testing facilities, road-scene data, and field engineering assistance. The company provided no direct financial funding for this study. Beyond the provision of these resources and field engineering assistance, the company had no role in the study design, annotation protocol, algorithm development, model training, experimental evaluation, statistical analysis, interpretation of the results, manuscript preparation, or the decision to submit the manuscript for publication. At the time of submission, no patent application, licensing agreement, formal product-integration agreement, or other commercialization agreement related to the method reported in this study existed. The remaining authors declare no competing interests.

References

1. Hu, Y.; Liu, G.; Chen, Z.; Guo, J. Object Detection Algorithm for Wheeled Mobile Robot Based on an Improved YOLOv4. *Appl. Sci.* **2022**, *12*, 4769. [CrossRef]
2. Su, Z.; Lin, X.; Sun, K.; Mao, S.; Zhou, X. Urban Trash Detection and Cleanliness Assessment Based on YOLOv7 for Autonomous Sweeper. In *Proceedings of the 2024 International Conference on Image Processing, Computer Vision and Machine Learning (ICICML)*; IEEE: Piscataway, NJ, USA, 2024. [CrossRef]
3. Wu, D.; Li, J.; Yang, W. STD-YOLOv8: A Lightweight Small Target Detection Algorithm for UAV Perspectives. *Electron. Res. Arch.* **2024**, *32*, 4563–4580. [CrossRef]
4. Qi, Y.; He, Y.; Qi, X.; Zhang, Y.; Yang, G. Dynamic Snake Convolution Based on Topological Geometric Constraints for Tubular Structure Segmentation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*; IEEE: Piscataway, NJ, USA, 2023; pp. 6070–6079. [CrossRef]
5. Lin, T.Y.; Dollár, P.; Girshick, R.; He, K.; Hariharan, B.; Belongie, S. Feature Pyramid Networks for Object Detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*; IEEE: Piscataway, NJ, USA, 2017; pp. 2117–2125. [CrossRef]

6. Liu, S.; Qi, L.; Qin, H.; Shi, J.; Jia, J. Path Aggregation Network for Instance Segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*; IEEE: Piscataway, NJ, USA, 2018; pp. 8759–8768. [CrossRef]
7. Tan, M.; Pang, R.; Le, Q.V. EfficientDet: Scalable and Efficient Object Detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*; IEEE: Piscataway, NJ, USA, 2020; pp. 10781–10790. [CrossRef]
8. Qiao, S.; Chen, L.C.; Yuille, A. DetectoRS: Detecting Objects with Recursive Feature Pyramid and Switchable Atrous Convolution. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*; IEEE: Piscataway, NJ, USA, 2021; pp. 10213–10224. [CrossRef]
9. Sunkara, R.; Luo, T. No More Strided Convolutions or Pooling: A New CNN Building Block for Low-Resolution Images and Small Objects. In *Proceedings of the Machine Learning and Knowledge Discovery in Databases: Research Track*; Amini, M.R., Canu, S., Fischer, A., Guns, T., Kralj Novak, P., Tsoumakas, G., Eds.; Lecture Notes in Computer Science; Springer: Cham, Switzerland, 2023; Volume 13715, pp. 443–459. [CrossRef]
10. Chen, J.; Kao, S.h.; He, H.; Zhuo, W.; Wen, S.; Lee, C.H.; Chan, S.H.G. Run, Don't Walk: Chasing Higher FLOPS for Faster Neural Networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*; IEEE: Piscataway, NJ, USA, 2023; pp. 12021–12031. [CrossRef]
11. Ouyang, D.; He, S.; Zhang, G.; Luo, M.; Guo, H.; Zhan, J.; Huang, Z. Efficient Multi-Scale Attention Module with Cross-Spatial Learning. In *Proceedings of the ICASSP 2023–2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*; IEEE: Piscataway, NJ, USA, 2023; pp. 1–5. [CrossRef]
12. Yang, L.; Zhang, R.Y.; Li, L.; Xie, X. SimAM: A Simple, Parameter-Free Attention Module for Convolutional Neural Networks. In *Proceedings of the 38th International Conference on Machine Learning, Virtual Event, 18–24 July 2021*; PMLR: Cambridge, MA, USA, 2021; Volume 139, pp. 11863–11874.
13. Jocher, G.; Qiu, J.; Liu, M.; Lyu, S.; Akyon, F.C.; Kalfaoglu, M.E. Ultralytics YOLO26: Unified Real-Time End-to-End Vision Models. *arXiv* **2026**, arXiv:2606.03748.
14. Ren, S.; He, K.; Girshick, R.; Sun, J. Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. In *Proceedings of the Advances in Neural Information Processing Systems*; Curran Associates, Inc.: Red Hook, NY, USA, 2015; Volume 28, pp. 91–99.
15. Redmon, J.; Divvala, S.; Girshick, R.; Farhadi, A. You Only Look Once: Unified, Real-Time Object Detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*; IEEE: Piscataway, NJ, USA, 2016; pp. 779–788. [CrossRef]
16. Ali, M.L.; Zhang, Z. The YOLO Framework: A Comprehensive Review of Evolution, Applications, and Benchmarks in Object Detection. *Computers* **2024**, *13*, 336. [CrossRef]
17. Wang, C.; He, W.; Nie, Y.; Guo, J.; Liu, C.; Wang, Y.; Han, K. Gold-YOLO: Efficient Object Detector via Gather-and-Distribute Mechanism. In *Proceedings of the Advances in Neural Information Processing Systems*; Curran Associates, Inc.: Red Hook, NY, USA, 2023; Volume 36.
18. Wang, C.Y.; Yeh, I.H.; Liao, H.Y.M. YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information. In *Proceedings of the Computer Vision–ECCV 2024*; Lecture Notes in Computer Science; Springer: Cham, Switzertland, 2024; Volume 15089, pp. 1–21. [CrossRef]
19. Wang, A.; Chen, H.; Liu, L.; Chen, K.; Lin, Z.; Han, J.; Ding, G. YOLOv10: Real-Time End-to-End Object Detection. In *Proceedings of the Advances in Neural Information Processing Systems*; Curran Associates, Inc.: Red Hook, NY, USA, 2024; Volume 37. [CrossRef]
20. Zhao, Y.; Lv, W.; Xu, S.; Wei, J.; Wang, G.; Dang, Q.; Liu, Y.; Chen, J. DETRs Beat YOLOs on Real-Time Object Detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*; IEEE: Piscataway, NJ, USA, 2024; pp. 16965–16974.
21. Peng, Y.; Li, H.; Wu, P.; Zhang, Y.; Sun, X.; Wu, F. D-FINE: Redefine Regression Task of DETRs as Fine-Grained Distribution Refinement. In Proceedings of the Thirteenth International Conference on Learning Representations (ICLR), Singapore, 24–28 April 2025.
22. Howard, A.; Sandler, M.; Chu, G.; Chen, L.C.; Chen, B.; Tan, M.; Wang, W.; Zhu, Y.; Pang, R.; Vasudevan, V.; et al. Searching for MobileNetV3. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*; IEEE: Piscataway, NJ, USA, 2019; pp. 1314–1324. [CrossRef]
23. Han, K.; Wang, Y.; Tian, Q.; Guo, J.; Xu, C.; Xu, C. GhostNet: More Features from Cheap Operations. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*; IEEE: Piscataway, NJ, USA, 2020; pp. 1580–1589. [CrossRef]
24. Mittal, P. A Comprehensive Survey of Deep Learning-Based Lightweight Object Detection Models for Edge Devices. *Artif. Intell. Rev.* **2024**, *57*, 242. [CrossRef]
25. Wang, A.; Chen, H.; Lin, Z.; Han, J.; Ding, G. RepViT: Revisiting Mobile CNN From ViT Perspective. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*; IEEE: Piscataway, NJ, USA, 2024; pp. 15909–15920.
26. Qin, D.; Leichner, C.; Delakis, M.; Fornoni, M.; Luo, S.; Yang, F.; Wang, W.; Banbury, C.; Ye, C.; Akin, B.; et al. MobileNetV4: Universal Models for the Mobile Ecosystem. In *Proceedings of the Computer Vision–ECCV 2024*; Lecture Notes in Computer Science; Springer: Cham, Switzerland, 2024; Volume 15098, pp. 78–96. [CrossRef]

27. Han, S.; Li, C.; Wang, X.; Li, X. YOLO-ULM: Ultra-Lightweight Models for Real-Time Object Detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*; IEEE: Piscataway, NJ, USA, 2026; pp. 18418–18427.
28. Wang, Q.; Wu, B.; Zhu, P.; Li, P.; Zuo, W.; Hu, Q. ECA-Net: Efficient Channel Attention for Deep Convolutional Neural Networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*; IEEE: Piscataway, NJ, USA, 2020; pp. 11534–11542. [CrossRef]
29. Russell, B.C.; Torralba, A.; Murphy, K.P.; Freeman, W.T. LabelMe: A Database and Web-Based Tool for Image Annotation. *Int. J. Comput. Vis.* **2008**, *77*, 157–173. [CrossRef]
30. Lin, T.Y.; Maire, M.; Belongie, S.; Hays, J.; Perona, P.; Ramanan, D.; Dollár, P.; Zitnick, C.L. Microsoft COCO: Common Objects in Context. In *Proceedings of the Computer Vision–ECCV 2014*; Lecture Notes in Computer Science; Springer: Cham, Switzerland, 2014; Volume 8693, pp. 740–755. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.